

Roguelike с LLM-агентами

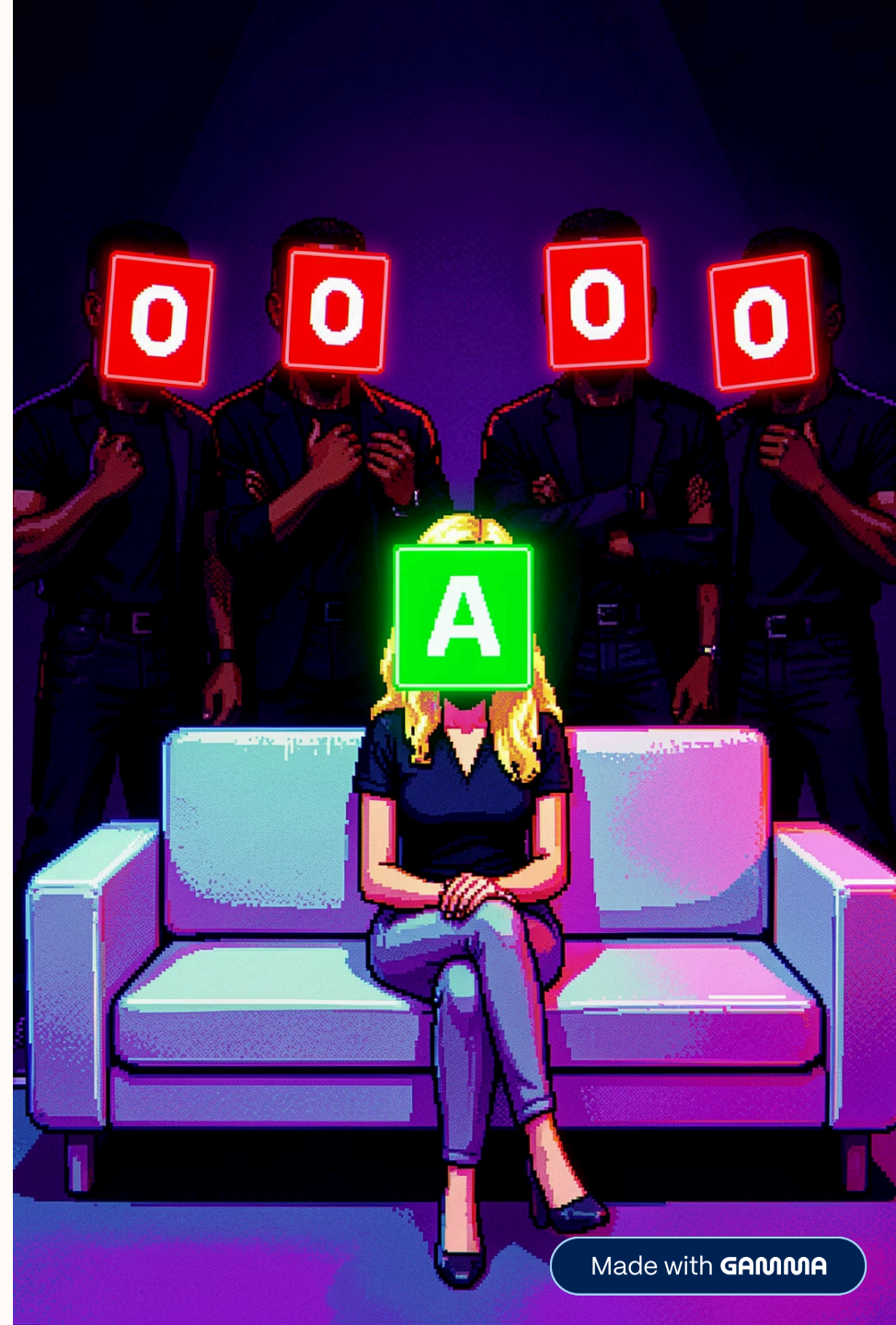
Д32 · Software Design · ИТМО MSE

Два автономных LLM-робота (**H** и **A**) соревнуются в **kill race** на процедурно сгенерированных уровнях. Чтобы сбежать из подземелья — набрать квоту убийств раньше соперника. Система поднимается одной командой `docker compose up`.

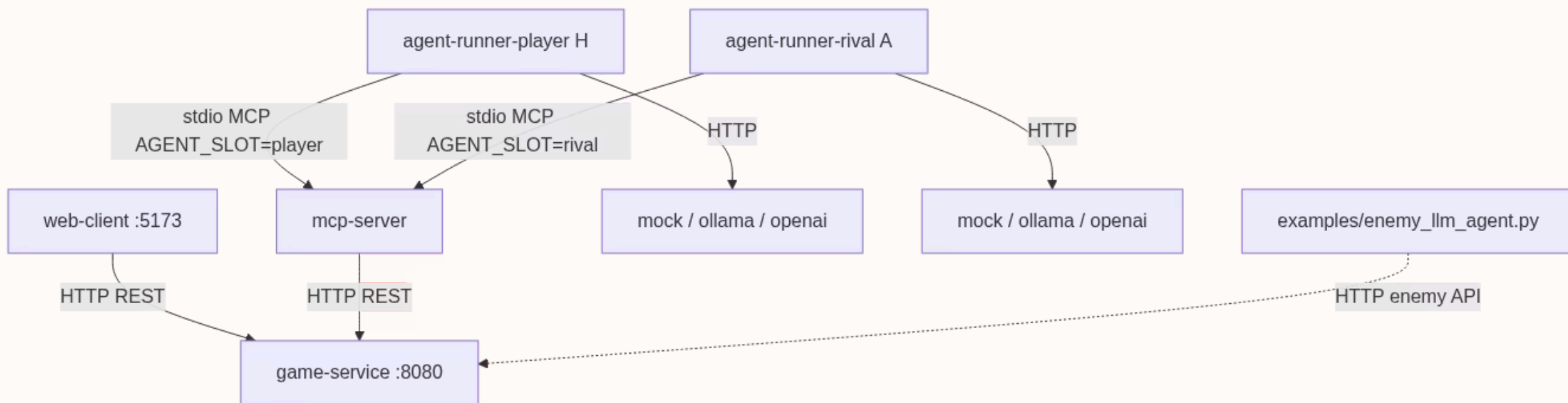
WEB-ГРАФИКА

МСП-ИНТЕГРАЦИЯ

3+ МИКРОСЕРВИСА



Архитектура системы



Границы ответственности чёткие: **game-service** не знает про LLM, **mcp-server** не знает про LLM, **agent-runner** не читает внутренний state — только MCP tools. MCP-сервер предоставляет 8 tools: `get_game_state`, `new_game`, `move`, `attack`, `scout_around`, `use_item`, `pickup_item`, `get_available_actions`.

GoF Паттерны

Strategy

game-service/src/State.cpp:50 — enemy_stats_for

Разные HP и урон для Goblin / Orc / Troll / Rat без if-else в каждом месте боя

State

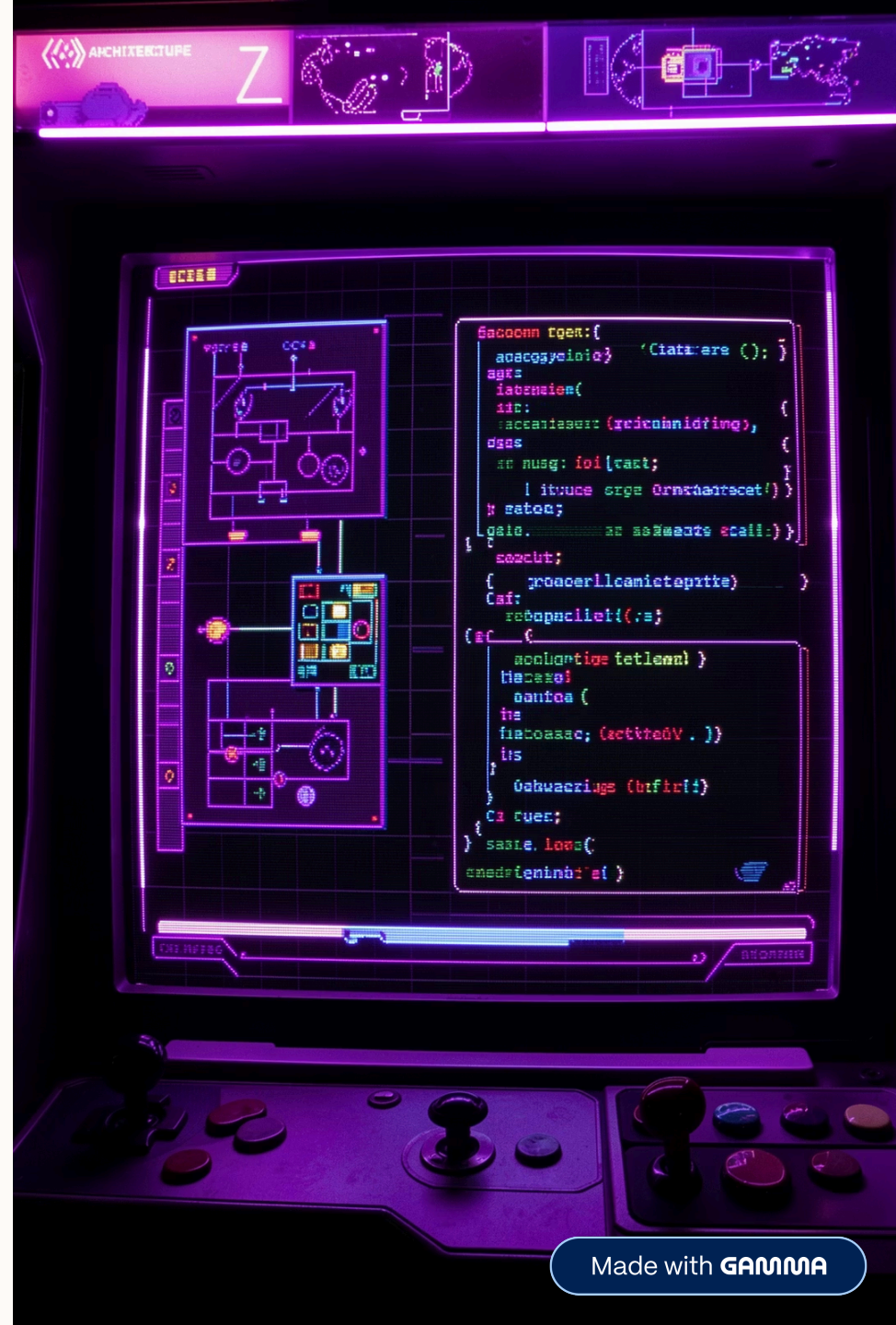
game-service/include/game_service/State.hpp:64 — GameState

Переходы running → level_complete / player_victory / enemy_victory

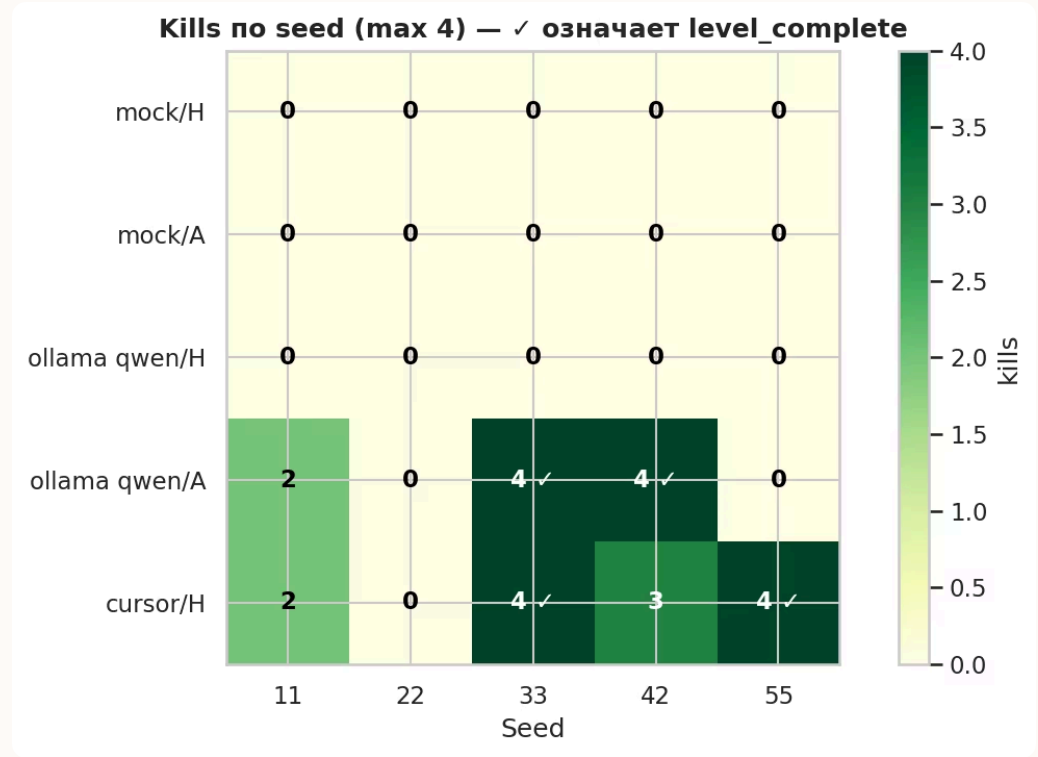
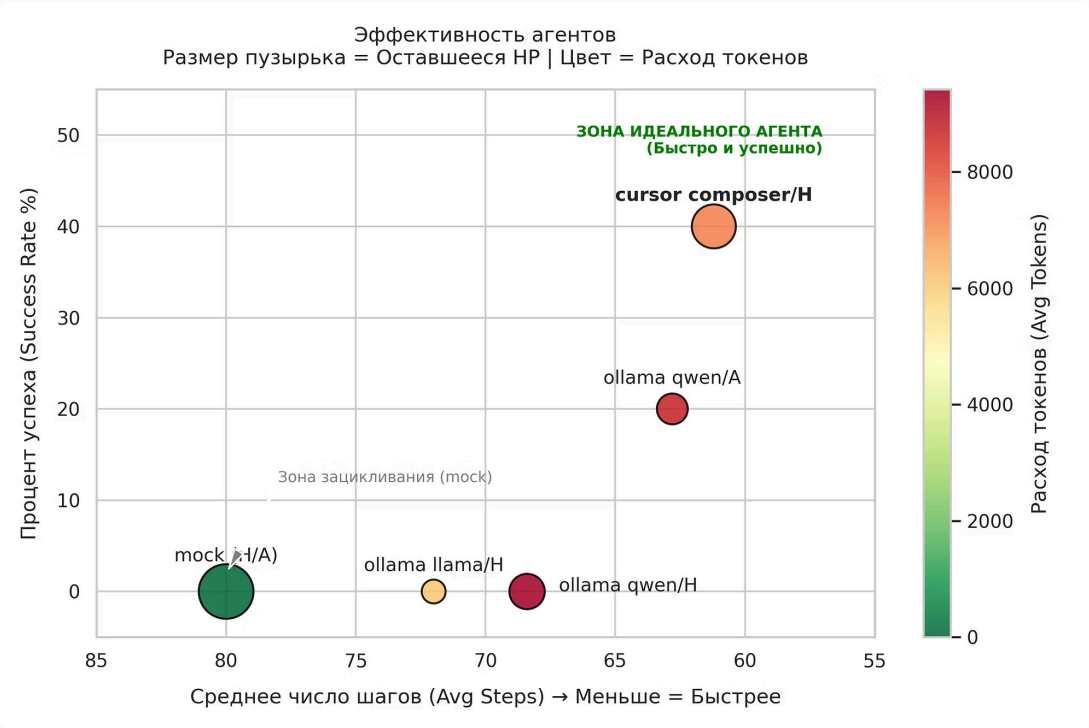
Factory

game-service/src/map/MapGenerator.cpp:114

BSP-генерация карты: комнаты и коридоры из одного входа



Сравнение агентов (eval)



Lessons Learned

→ AI как pair-programmer, не black box

Cursor/ChatGPT/Copilot участвовали в инфраструктуре и документации. Ядро game-service и agent loop — вручную. На защите команда понимает свой код.

→ Graceful degradation **обязателен**

Retry с backoff → fallback на mock-стратегию; при исчерпании `MAX_TOTAL_TOKENS` — пауза раунда. Anti-loop guards в `llm_client.hpp`.

→ Mock-агент — **необходимый** baseline

`LLM_PROVIDER=mock` — ходит по кругу, 0 kills, 0% success. Критичен для eval и fallback при ошибке Ollama.

